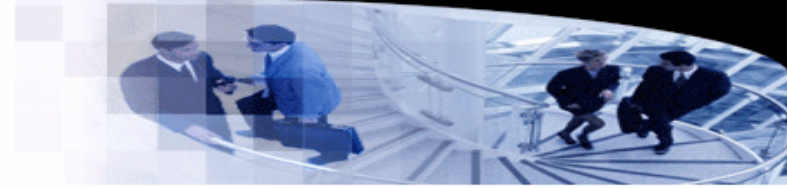




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

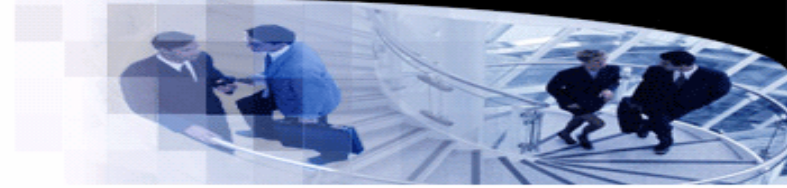
Lecture 19. Intermediate Code Generation

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

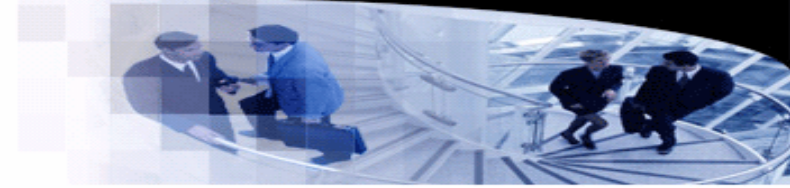
Intermediate Code Generation



- Translation between intermediate representations

- Compilers have multiple IRs.
- High-level IRs: AST
- Mid-level IRs: CFG and DAG
- Low-level IRs: Quadruple and RTL
- The compilation from a source program to a target program can be considered as a sequence of transformations, where each step translates a higher-level IR into a lower-level IR.
- In this lecture, we demonstrate some of these transformations.

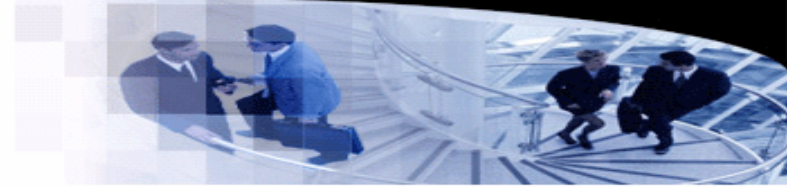
Generating IRs from Source Codes



- Generating ASTs from source programs
 - It is easy to construct an AST in a bottom-up syntax-directed parser.

Productions	Semantic Rules	Semantic Actions
$E \rightarrow E^1 + T$	$E.ast = node(+)$ $E.ast[0] = E^1.ast$ $E.ast[1] = T.ast$	$\{E.ast$ $\quad := createTree(+, E^1.ast, T.ast); \}$
$E \rightarrow T$	$E.ast = T.ast$	$\{E.ast := T.ast; \}$
$T \rightarrow T^1 * F$	$T.ast = node(*)$ $T.ast[0] = T^1.ast$ $T.ast[1] = F.ast$	$\{T.ast$ $\quad := createTree(*, T^1.ast, F.ast); \}$
$T \rightarrow F$	$T.ast = F.ast$	$\{T.ast := F.ast; \}$
$F \rightarrow (E)$	$F.ast = E.ast$	$\{F.ast := E.ast; \}$
$F \rightarrow id$	$F.ast = node(id.text)$	$\{F.ast := createTree(id.text); \}$

Generating IRs from Source Codes

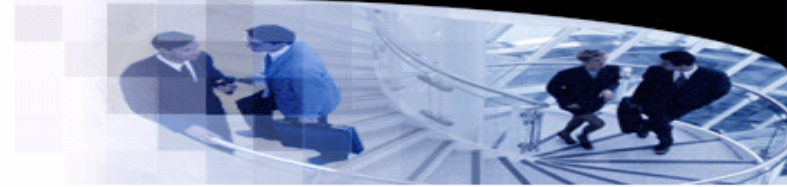


- Generating ASTs from source programs

- An ocaml yacc implementation of generating ASTs from arithmetic expressions.
- Grammar
 - $Term \rightarrow AppTerm \mid if\ Term\ then\ Term\ else\ Term$
 - $AppTerm \rightarrow ATerm \mid succ\ ATerm \mid pred\ ATerm \mid iszero\ ATerm$
 - $ATerm \rightarrow (Term) \mid true \mid false \mid intv$
- AST

```
type term =  
  | TmTrue of info  
  | TmFalse of info  
  | TmIf of info * term * term * term  
  | TmZero of info  
  | TmSucc of info * term  
  | TmPred of info * term  
  | TmIsZero of info * term
```

Generating IRs from Source Codes



- Generating ASTs from source programs

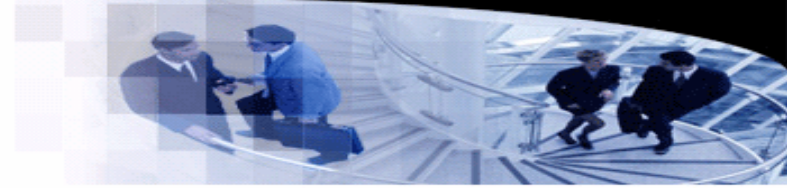
- An ocaml yacc implementation of generating ASTs from arithmetic expressions.
- Grammar
 - $Term \rightarrow AppTerm \mid \text{if } Term \text{ then } Term \text{ else } Term$
 - $AppTerm \rightarrow ATerm \mid \text{succ } ATerm \mid \text{pred } ATerm \mid \text{iszero } ATerm$
 - $ATerm \rightarrow (Term) \mid \text{true} \mid \text{false} \mid \text{intv}$
- Translation schemes in ocaml yacc

```
Term :  
  AppTerm  
  | { $1 }  
  | IF Term THEN Term ELSE Term  
  | { TmIf($1, $2, $4, $6) }
```

```
AppTerm :  
  ATerm  
  | { $1 }  
  | SUCC ATerm  
  | { TmSucc($1, $2) }  
  | PRED ATerm  
  | { TmPred($1, $2) }  
  | ISZERO ATerm  
  | { TmIsZero($1, $2) }
```

```
ATerm :  
  LPAREN Term RPAREN  
  | { $2 }  
  | TRUE  
  | { TmTrue($1) }  
  | FALSE  
  | { TmFalse($1) }  
  | INTV  
  | { let rec f n = match n with  
      | 0 -> TmZero($1.i)  
      | n -> TmSucc($1.i, f (n-1))  
      in f $1.v }
```

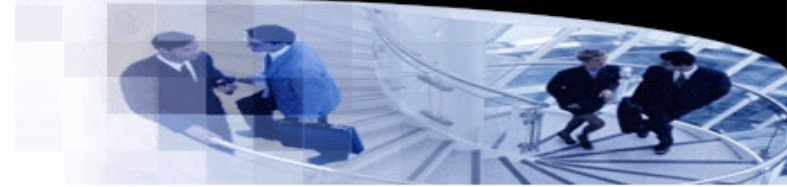
Generating IRs from Source Codes



- Generating quadruples from source programs
 - It is also possible to generate low-level IRs directly from the source program, although it is not recommended.
 - Quadruples are designed for virtual machines.
 - The key of generating quadruples is using virtual registers.
 - In this example, the virtual machine is supposed to have infinite number of registers.
 - The function `emit` is used to generate a quadruple.

Productions	Sematic Actions
$S \rightarrow id := E$	{ emit(<code>:=</code> , E.r, <code>_</code> , id.text; }
$E \rightarrow E^1 + T$	{ E.r := newRegister(); emit(<code>+</code> , E.r, E ¹ .r, T.r; }
$E \rightarrow T$	{ E.r := T.r; }
$T \rightarrow T^1 * F$	{ T.r := newRegister(); emit(<code>*</code> , T.r, T ¹ .r, F.r; }
$T \rightarrow F$	{ T.r := F.r; }
$F \rightarrow (E)$	{ F.r := E.r; }
$F \rightarrow id$	{ F.r := newRegister(); emit(<code>load</code> , F.r, <code>_</code> , id.text; }

Translating Declarations

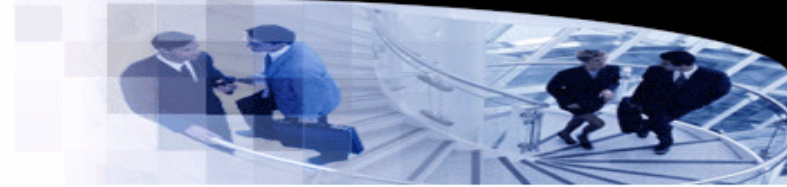


- **Symbol tables**

- In procedural languages, a declaration of variable binds a name to a certain memory location.
- The translation should store these bindings in a **symbol table**.
- Let us consider the declarations in a function.
- The declared variables are locations in the function's stack frame.

Productions	Semantic Actions
$P \rightarrow SD$	
$S \rightarrow \epsilon$	{ offset := 0; }
$D \rightarrow D; D$	
$D \rightarrow id:T$	{ enterSymbol(id.text, T.type, offset); offset := offset + T.size; }
$T \rightarrow int$	{ T.type := integer; T.size := 4; }
$T \rightarrow float$	{ T.type := float; T.size := 8; }
$T \rightarrow T^1[num]$	{ T.type := array(T^1 .type); T.size := T^1 .size * num.value; }
$T \rightarrow T^1 *$	{ T.type := pointer(T^1 .type); T.size := 4; }

Translating Declarations



- **Scopes**

- Declarations have their own scopes.
- In this example, we use symbol tables to represent scopes.
- Nested scopes are implemented with nested symbol tables.
- The pointer ct records the current symbol table where new declarations are entered.

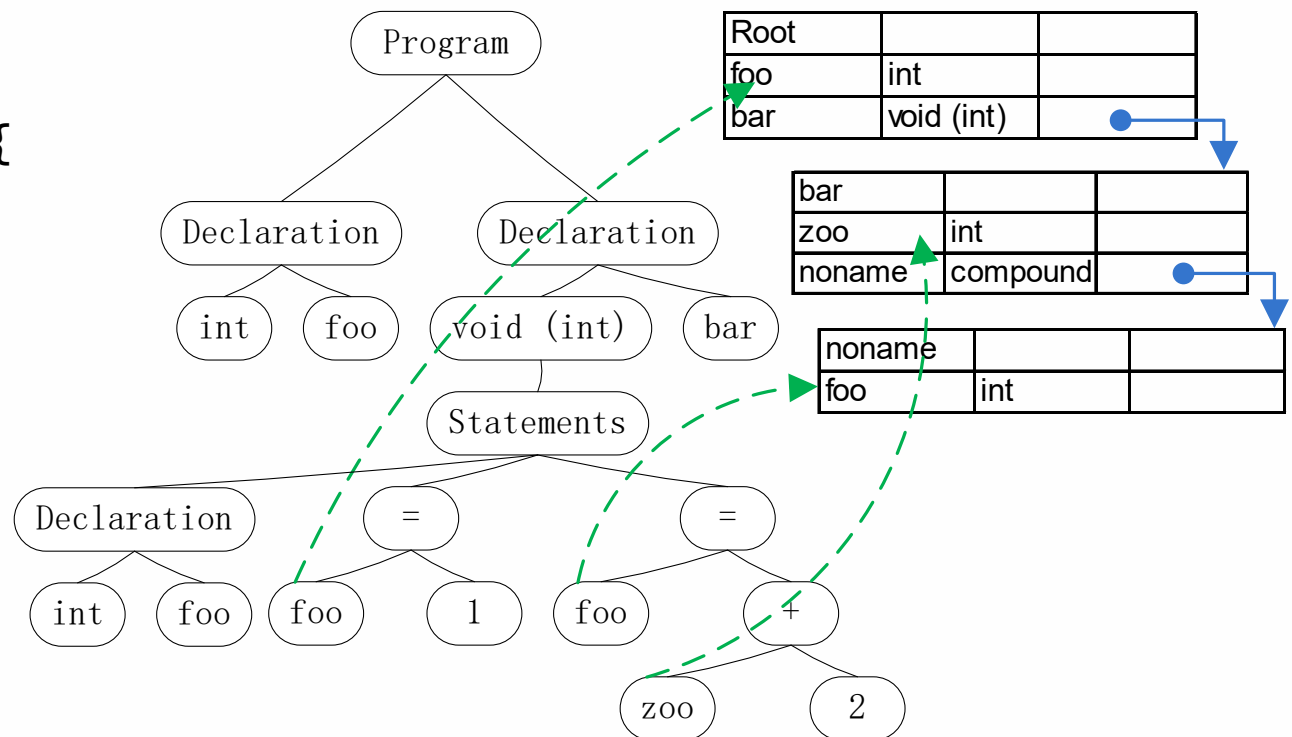
Productions	Sematic Actions
$D \rightarrow T\ id\ D'(PL)\ \{ Stmts \} D''$	
$D' \rightarrow \epsilon$	{ push(ct); ct := newTable(ct, \$0.text); }
$D'' \rightarrow \epsilon$	{ ct := pop(); }
...	
$D \rightarrow id:T$	{ entersymbol(ct, id.text, T.type, offset); }
...	
$F \rightarrow id$	{ F.ent := lookup(ct, id.text); }

Translating Declarations

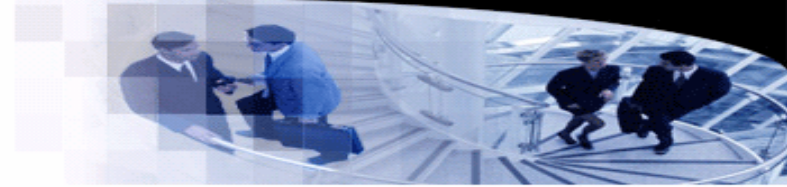
- **Scopes**

- Example:

- `int foo;`
- `void bar(int zoo) {`
- `foo = 1;`
- `{`
- `int foo;`
- `foo = zoo + 2;`
- `}`
- `}`



Type Checking



- **Typed AST**

- The original AST generated by the parser is untyped, which means that some nodes have not been typed.
- The parser will perform the semantic analysis on the AST to ensure the program is well-typed.
- Thus, the AST becomes an typed AST, where each typable node has a type.
- Several type checking algorithms may be performed by the parser.

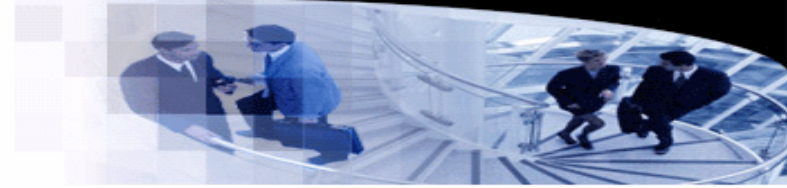
- **Type synthesis**

- A bottom-up type checking algorithm that construct the type of a parent node, based on the types of its children.

- **Type inference**

- Infer the type of a structure, based on how the structure is used.

Type Checking

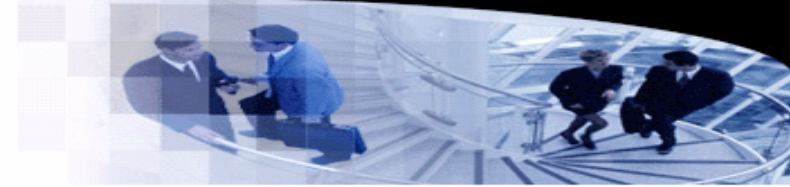
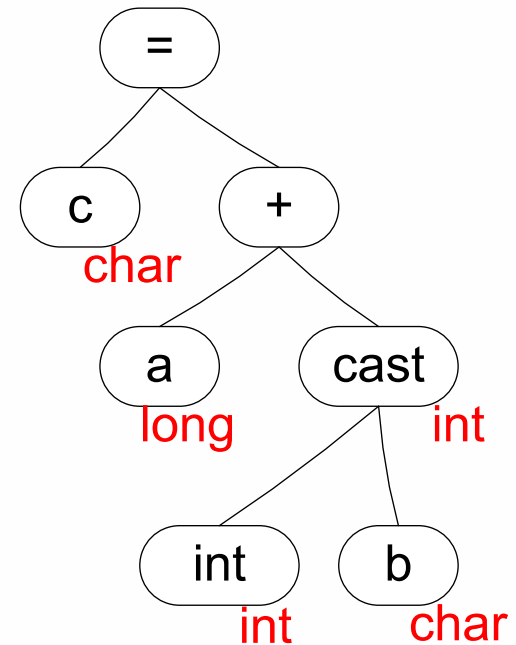
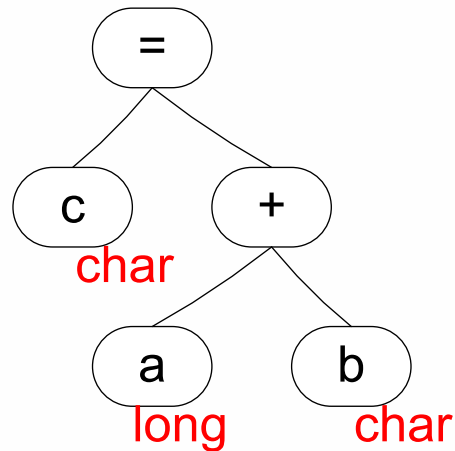


- **Type casting**
 - Sometimes the type of an operand should be converted into another type to satisfy the semantics of the operator.
- **Implicit type casting**
 - These casting operations are inserted by compilers automatically.
- **Explicit type casting**
 - They are written by programmers.

Type Checking

- Example of C type checking

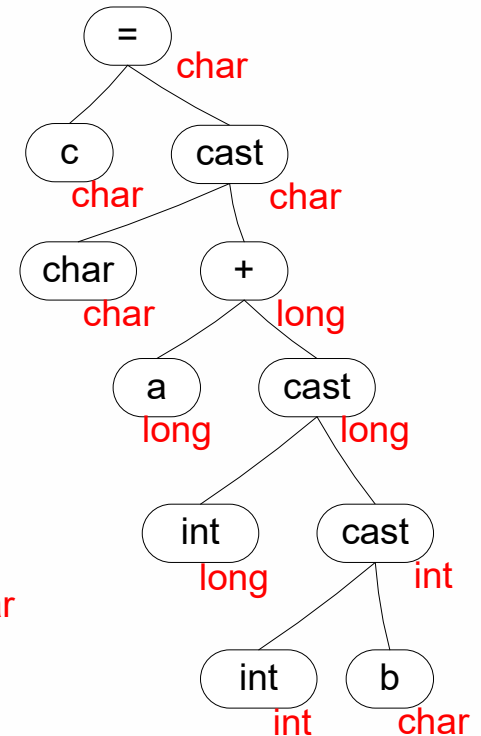
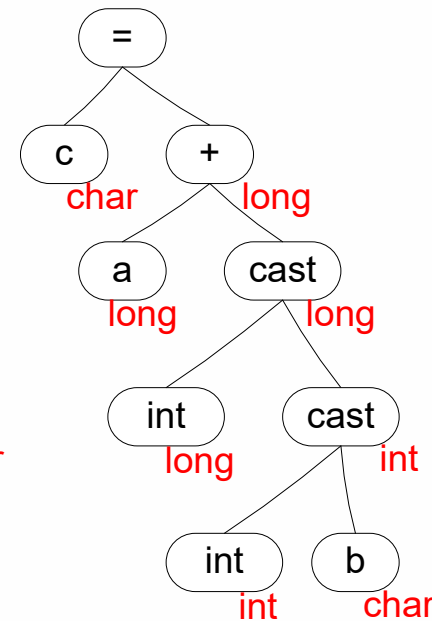
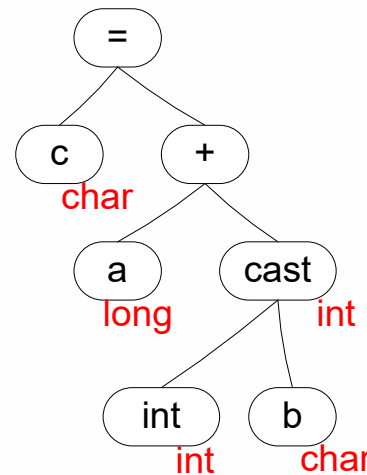
- An input program and its AST
 - `long a;`
 - `char b, c;`
 - `c = a + b;`
- The original AST is untyped.
- Type casts are inserted by compilers during type checking.



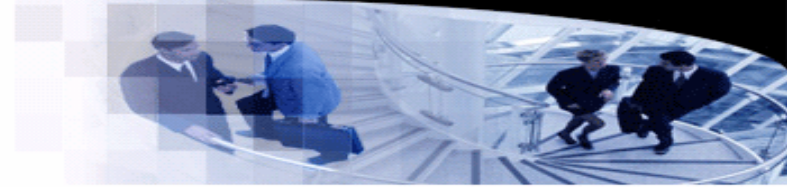
Type Checking

- Example of C type checking

- An input program and its AST
 - `long a;`
 - `char b, c;`
 - `c = a + b;`
- The original AST is untyped.
- Type casts are inserted by compilers during type checking.



Generating Sequential Codes

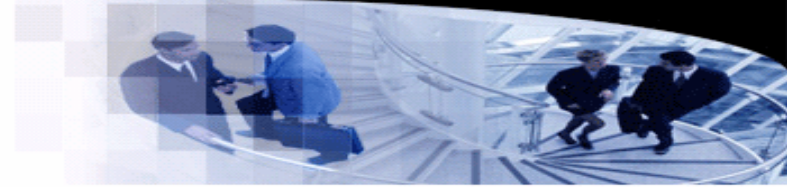


- **Generating quadruples from typed AST**

- Given a typed AST, it is straightforward to generate quadruples by using an bottom-up algorithm.

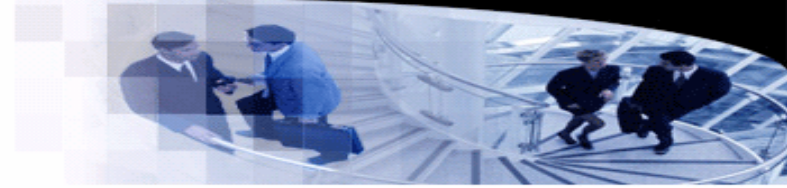
```
• t = PostOrder(ast);
• while t != nil do
•   if IsBinary(t)
•     t.code.add(t.children[0].code);
•     t.code.add(t.children[1].code);
•     t.result := newRegister();
•     t.code.add(t.result, t.op, t.children[0].result
•               t.children[1].result);
•   else if IsIdentifier(t)
•     t.result = newRegister();
•     t.code.add(t.result, "load", t.text);
•   endif
•   t = PostOrder(ast);
• endwhile
```

Generating Sequential Codes



- Generating quadruples from control-flow statements
 - Source code:
 - `if (e) s1 else s2;`
 - Quadruples:
 - L1:
 - `%1 = e;`
 - `jzero %1, L2;`
 - `s1;`
 - `jmp L3;`
 - L2:
 - `s2;`
 - L3:

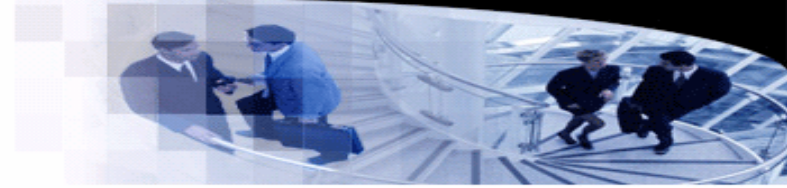
Generating Sequential Codes



- Short-circuit evaluation

- When Boolean expressions are used in control-flow statements, it is more efficient to translate them into branch statements.
- If the semantics of a language allows lazy evaluation of Boolean expressions, a short-circuit evaluation strategy can be used on Boolean expressions.
- Example
 - `if (p && p->x > 0)`
 - `q += p->x;`
 - When `p` is zero, it is unnecessary to evaluate `p->x > 0`.

Generating Sequential Codes



- **Short-circuit evaluation**

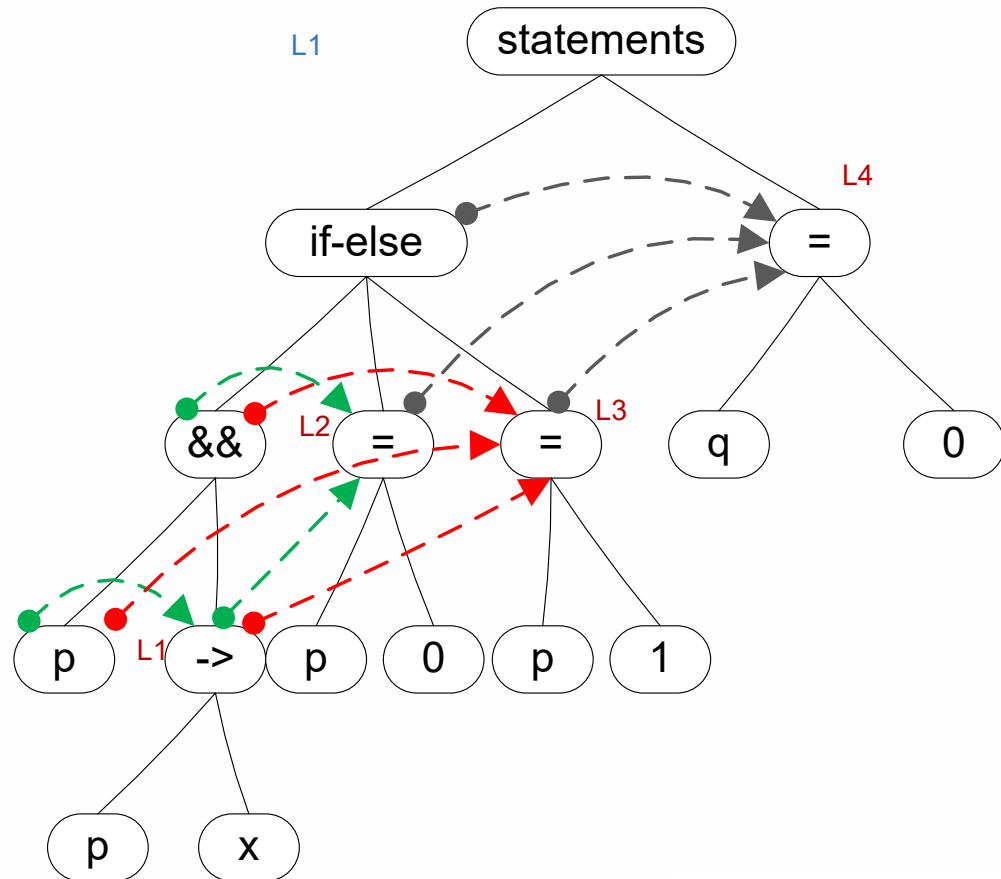
- For each Boolean expression, add two attributes (i.e., `tentry` and `fentry`) to the nodes.
- `tentry` and `fentry` indicate the target labels of the branch statements when the expression evaluates to true or false.
- These entries will be translated into branch statements later.
- For other statements, there is a `nentry` indicating the next statement on the control flow.
- A preorder traversal is performed to calculate all these attributes.
- A post order traversal is then performed to generate quadruples according to these attributes.

Generating Sequential Codes

- Short-circuit evaluation

- Example

- `if (p && p->x)`
- `p = 0;`
- `else`
- `p = 1;`
- `q = 0;`

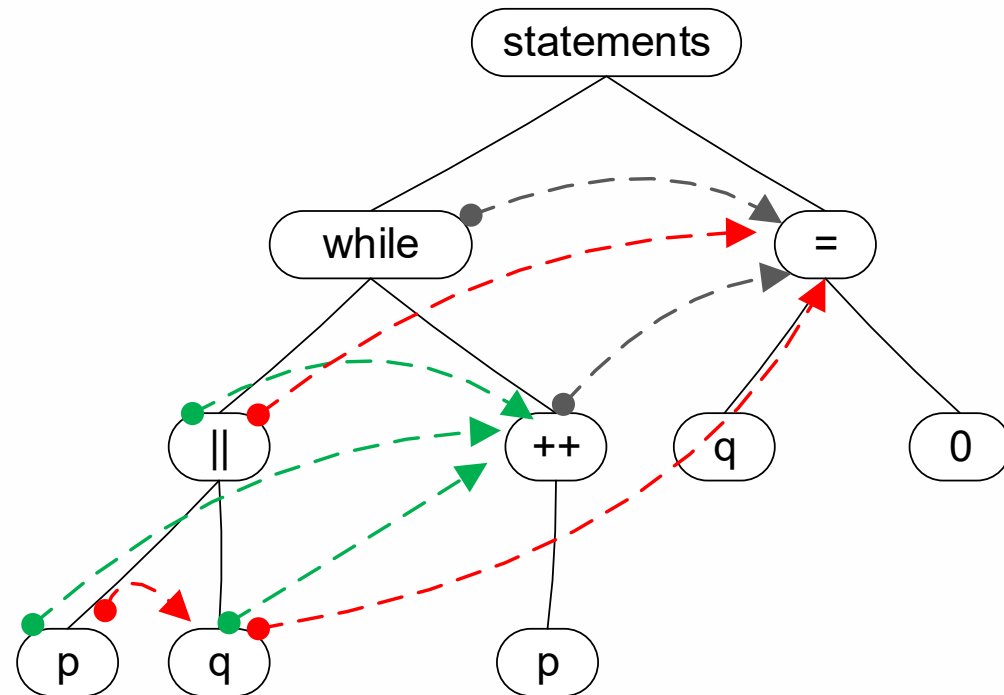


Generating Sequential Codes

- Short-circuit evaluation

- Example

- `while (p || q)`
- `p ++;`
- `q = 0;`



Code Generation via Recursive Descent Parser

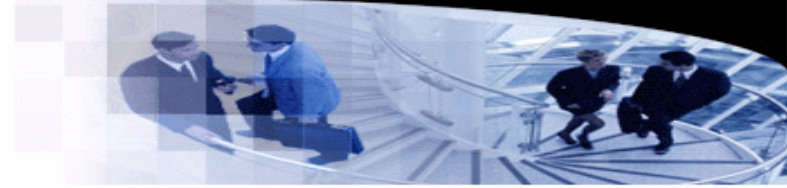
- Generating AST

- Consider the following grammar.

- $E \rightarrow \text{if } E \text{ then } E \text{ else } E$
- $E \rightarrow TE'$
- $E' \rightarrow +TE' | \epsilon$
- $T \rightarrow FT'$
- $T' \rightarrow * FT' | \epsilon$
- $F \rightarrow (E) | a$

- AST definition:

```
type term =  
  | Identifier  
  | Plus of term * term  
  | Mul of term * term  
  | IfElse of term * term * term;;
```



```
let print_term t =  
  let rec print ident t =  
    print_string (String.make ident ' ');  
    match t with  
    | Identifier -> print_string "a\n"  
    | Plus(t1, t2) ->  
      print_string "+\n";  
      print (ident + 4) t1;  
      print (ident + 4) t2  
    | Mul(t1, t2) ->  
      print_string "*\n";  
      print (ident + 4) t1;  
      print (ident + 4) t2  
    | IfElse(t1, t2, t3) ->  
      print_string "if-else\n";  
      print (ident + 4) t1;  
      print (ident + 4) t2;  
      print (ident + 4) t3  
  in print 0 t;
```

Code Generation via Recursive Descent Parser

- Generating AST

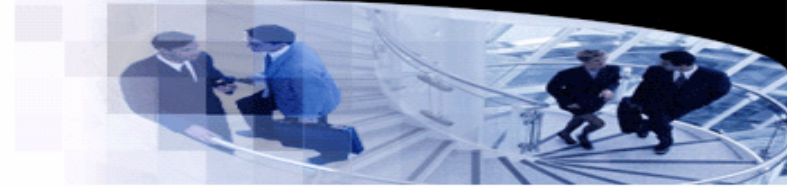
- Handle the following productions.

- $E \rightarrow \text{if } E \text{ then } E \text{ else } E$
- $E \rightarrow TE'$

```
let rec parse_e input =  
  match input with  
  | 'a'::xs | '('::xs ->  
    let (t1, s1) = parse_t input in  
    parse_e1 t1 s1  
  | 'i'::xs -> parse_if xs  
  | _ -> raise (Failure "Error e")
```

- The AST of T is passed to E' as an argument.

```
and parse_if input =  
  match input with  
  | '('::xs | 'a'::xs ->  
    let (t1, s1) = parse_e input in  
    (match s1 with  
    | 't'::x1 ->  
      let (t2, s2) = parse_e x1 in  
      (match s2 with  
      | 'e'::x2 ->  
        let (t3, s3) = parse_e x2 in  
        (IfElse(t1, t2, t3), s3)  
      | _ -> raise (Failure "Error if")  
      )  
    | _ -> raise (Failure "Error if")  
    )  
  | _ -> raise (Failure "Error if")  
;;
```



Code Generation via Recursive Descent Parser

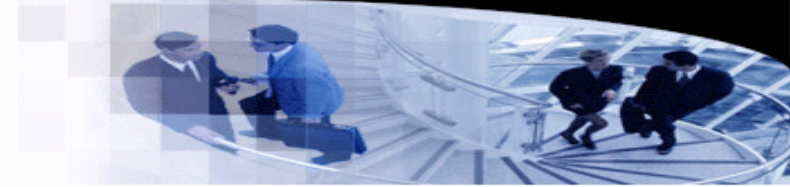
- Generating AST

- Handle the following productions.

- $E' \rightarrow +TE'$
- Create a new AST with the root '+'. The left child is the argument $t1$. The right child is AST of T .

```
and parse_e1 t1 input =  
  match input with  
  | '+'::xs ->  
    let (t2, s2) = parse_t xs in  
    parse_e1 (Plus(t1, t2)) s2  
  | '$'::xs | ')'::xs | 't'::xs | 'e'::xs -> (t1, input)  
  | _ -> raise (Failure "Error e1")
```

- $E' \rightarrow \epsilon$
- Just return the AST in the argument.



Code Generation via Recursive Descent Parser

- Generating AST

- Handle the following productions.

- $T \rightarrow FT'$
- $T' \rightarrow * FT' | \epsilon$
- $F \rightarrow (E) | a$

```
and parse_t input =  
  match input with  
  | 'a'::xs | '('::xs ->  
    let (t1, s1) = parse_f input in  
    parse_t1 t1 s1  
  | _ -> raise (Failure "Error t")
```

- Wrap them together.

```
let parse input =  
  let explode s = List.init (String.length s) (String.get s) in  
  fst (parse_e (explode input));;
```

```
and parse_t1 t1 input =  
  match input with  
  | '*'::xs ->  
    let (t2, s2) = parse_f xs in  
    parse_t1 (Mul(t1, t2)) s2  
  | '$'::xs | ')'::xs | '+'::xs | 't'::xs | 'e'::xs -> (t1, input)  
  | _ -> raise (Failure "Error t1")
```

```
char list -> term * char list  
and parse_f input =  
  match input with  
  | 'a'::xs -> (Identifier, xs)  
  | '('::xs ->  
    let (t1, s1) = parse_e xs in  
    (match s1 with  
    | ')'::s2 -> (t1, s2)  
    | _ -> raise (Failure "Error f")  
    )  
  | _ -> raise (Failure "Error f")
```

